FORMATO DE DESARROLLO E INTEGRACIONES

API DATACREDITO

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, martes, 15 de septiembre de 2026

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios.....**¡Error! Marcador no definido.**
2. CASOS DE PRUEBAS UNITARIAS 16

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

En el presente documento se describe técnicamente como fue desarrollada la API que interviene en el proceso de originación de cupo. Los pasos claves de este requerimiento incluye:

a) Consumo del servicio web historia de crédito+: La API se conecta de manera segura al servicio proporcionado por Datacrédito Experian donde se obtendrá los datos crediticios de personas naturales según los datos de la solicitud de entrada.
b) Procesamiento de datos: Una vez se recibe los datos del servicio web se lleva a cabo una normalización de la información recibida y el procesamiento de esta para otorgar una decisión de aprobación o negación del cupo.
c) Retorno de información crediticia: La API responde a las solicitudes de los usuarios proporcionando la información crediticia detallada y fácilmente interpretable para quien realiza la solicitud.

Este requerimiento se basa en la necesidad de acceder de manera eficiente a la información crediticia de individuos para tomar una decisión puntual en el proceso de originación en el aplicativo de credi10, teniendo en cuenta la escalabilidad que este pueda tener a futuro.

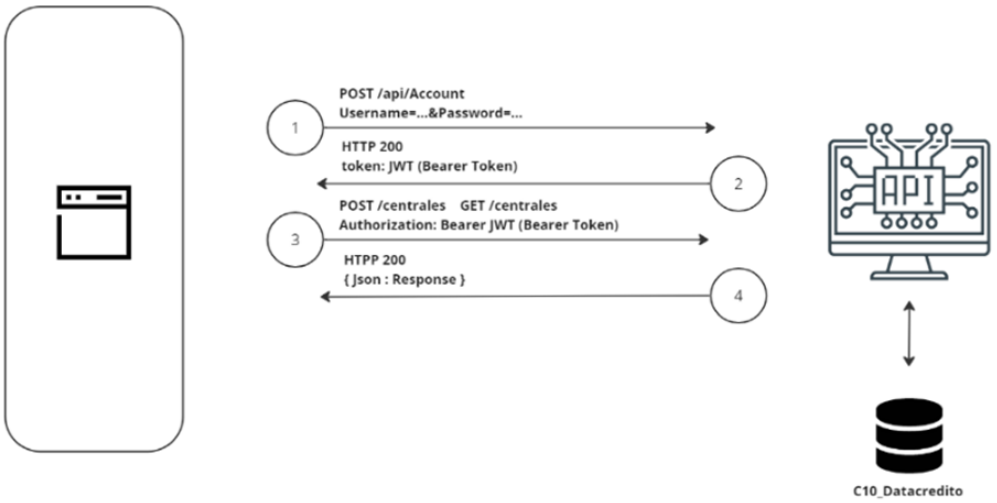
1.2 Detallar componentes existentes

SOLUCION	
Datacredito.Integrator.Api	Proyecto que proporciona un conjunto de servicios a otras aplicaciones a través de una interfaz de programación de aplicaciones (API).
Datacredito.Integrator.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de consumir el servicio web, procesarlo y entregar la información para su posterior envió al usuario final. Este proyecto se integra con el proyecto API y el proyecto DAL.
Datacredito.Integrator.DAL	Proyecto encargado de tener el acceso y la interacción a la base de datos C10_Datacredito de la solución
Web Service Produccion Historia de crédito +	URL CONSUMO: https://servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus

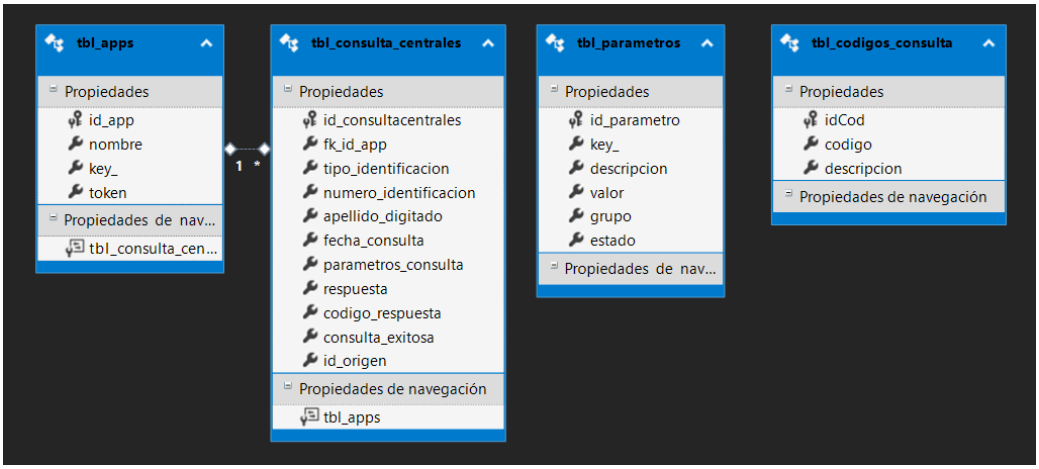
	URL WSDL: https://servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus?wsdl
Web Service Pruebas Historia de crédito +	URL CONSUMO: https://demo-servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus URL WSDL: https://demo-servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus?wsdl
BASE DE DATOS	
C10_Datacredito.dbo.tbl_apps	Tablas
C10_Datacredito.dbo.tbl_codigos_consulta	Tablas
C10_Datacredito.dbo.tbl_consulta_centrales	Tablas
C10_Datacredito.dbo.tbl_parametros	Tablas

1.3 Detallar funcionalidades

1.3.1 Diagrama de Arquitectura



1.3.2 MER



1.3.3 Parametrizaciones

La aplicación cuenta con 18 parámetros que pueden cambiar dependiente del ambiente que se este utilizando o las renovaciones de claves y productos que se tengan cada año de cara al proveedor Datacrédito Experian. Por lo que es importante tener estos parámetros actualizados. Los parámetros son:

Llave parámetros	Descripción	Valor Ambiente Productivo	Valor Ambiente Pruebas
AmmountOKKey	Monto Aprobado Datacrédito	CUPO	CUPO
CentralesUrl	Url Servicio Datacrédito	https://servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus	https://demo-servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus
ClientPassword	Contraseña del cliente de datacredito	89UPQ	99TPF
ClientProduct	Producto ofrecido por Datacredito Experian (DICTUM)	64	64
ClientUser	Usuario del cliente	805003626	805003626
DenialCauseKey	Causa de denegación	CAUSAL	CAUSAL

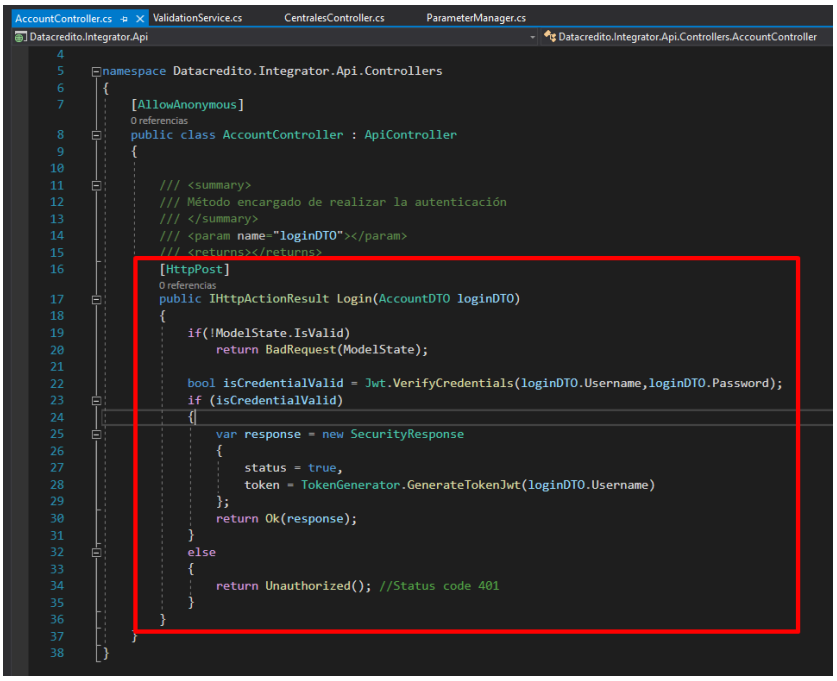
DnsIdentity	Identidad del servicio que se consume	servicesesb.datacredito.com.co	dev-servicesesb.datacredito.com.co
ServicePassword	Password del Servicio	Fernando2020	Oscar2021
ServiceUser	Usuario servicio	2-805003626	2-805003626
StatusOKKey	Estado Aprobado	DECISION	DECISION
StatusOKValue	Valor Aprobado	APROBADO	APROBADO
ClientSSL	Certificado SSL del cliente compartido por Data Credito Experian	C:/CertificadoCredi10/certificadoAmbienteProductivo.cer	C:/CertificadoCredi10/certificadoNuevo.cer
ServerSSL	Clave privada para consumir el	C:/CertificadoCredi10/credi10202030414.p12	C:/CertificadoCredi10/credi10 20230414.p12

	servicio		
SSLPassword	Contraseña de la clave privada	xxxxxxx	xxxxxxx
JWT_SECRET_KEY	Llave secreta para el JWT	47590b37-0b12-45a9-9e37-189b930b589d	47590b37-0b12-45a9-9e37-189b930b589d
JWT_AUDIENCE_TOKEN	Cliente con autorizacion para consumir la API	http://192.168.0.xxx/	http://192.168.0.xxx/
JWT_ISSUER_TOKEN	Cliente que podra realizar peticiones a la API	http://192.168.0.xxx/	http://192.168.0.xxx/
JWT_EXPIRE_MINUTES	Tiempo de expiracion para el consumo de la API	5	5

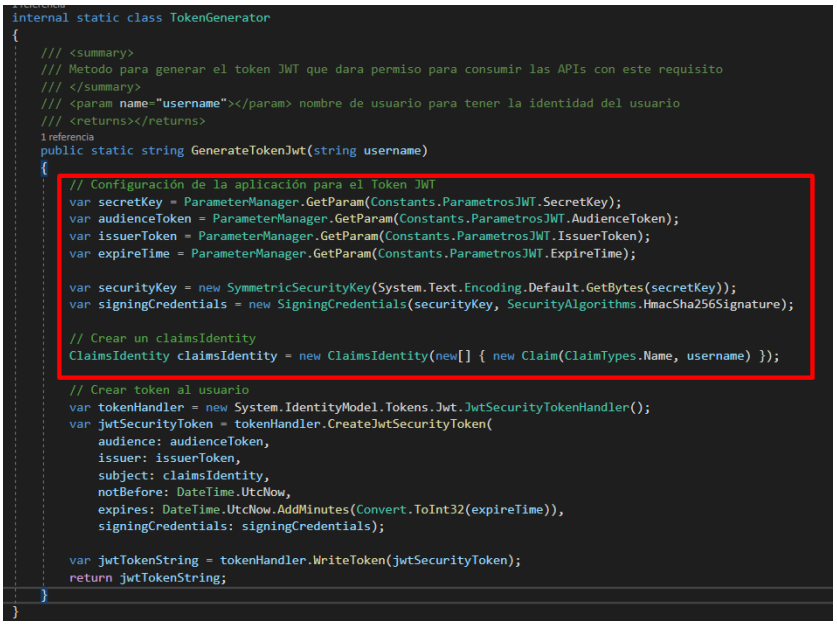
Nota: Algunos de estos valores se renuevan año tras año en los dos ambientes, por lo que estos valores solo son referencia.

1.3.4 Proceso

El primer paso para poder consumir la API consiste en la autenticación mediante una solicitud POST. Este proceso implica recibir un par de credenciales de usuario, compuesto por el nombre de usuario (username) y la contraseña (password). Si las credenciales proporcionadas son válidas, la API genera y devuelve un token de acceso JSON Web Token (JWT). Este token se utiliza posteriormente para autorizar y autenticar las solicitudes subsiguientes del usuario. En caso de que las credenciales recibidas en la solicitud sean inválidas, la API responde con un código de estado HTTP 401 (No Autorizado), indicando que la autenticación ha fallado. Este mecanismo de autenticación basado en JWT contribuye a la seguridad de la API al proporcionar un método eficaz para la gestión de sesiones y la protección de recursos mediante la verificación de la identidad del usuario.



En la verificación de las credenciales recibidas la aplicación consulta si el usuario existe en la base de datos de la aplicación y si la contraseña recibida pertenece a dicho usuario. Si las credenciales son válidas se inicia el proceso de generar el JWT para su posterior envió como respuesta a la solicitud.



```
internal static class TokenGenerator
{
    /// <summary>
    /// Metodo para generar el token JWT que dara permiso para consumir las APIs con este requisito
    /// </summary>
    /// <param name="username"></param> nombre de usuario para tener la identidad del usuario
    /// <returns></returns>
    1 referencia
    public static string GenerateTokenJwt(string username)
    {
        // Configuración de la aplicación para el Token JWT
        var secretKey = ParameterManager.GetParam(Constants.ParametrosJWT.SecretKey);
        var audienceToken = ParameterManager.GetParam(Constants.ParametrosJWT.AudienceToken);
        var issuerToken = ParameterManager.GetParam(Constants.ParametrosJWT.IssuerToken);
        var expireTime = ParameterManager.GetParam(Constants.ParametrosJWT.ExpireTime);

        var securityKey = new SymmetricSecurityKey(System.Text.Encoding.Default.GetBytes(secretKey));
        var signingCredentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256Signature);

        // Crear un claimsIdentity
        ClaimsIdentity claimsIdentity = new ClaimsIdentity(new[] { new Claim(ClaimTypes.Name, username) });

        // Crear token al usuario
        var tokenHandler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var jwtSecurityToken = tokenHandler.CreateJwtSecurityToken(
            audience: audienceToken,
            issuer: issuerToken,
            subject: claimsIdentity,
            notBefore: DateTime.UtcNow,
            expires: DateTime.UtcNow.AddMinutes(Convert.ToInt32(expireTime)),
            signingCredentials: signingCredentials);

        var jwtTokenString = tokenHandler.WriteToken(jwtSecurityToken);
        return jwtTokenString;
    }
}
```

El método *GenerateTokenJwt* funciona de la siguiente manera:

- Obtención de Parámetros: Se obtienen los parámetros necesarios para la creación del JWT desde la configuración de la aplicación. Estos parámetros incluyen la clave secreta (secretKey), la audiencia (audienceToken), el emisor (issuerToken) y el tiempo de expiración (expireTime), más adelante se detallará cada uno de estos parámetros.

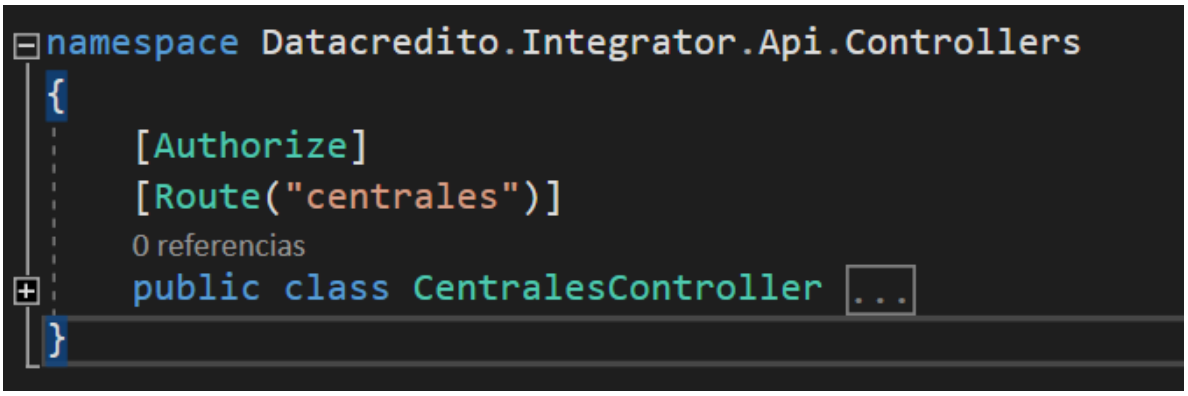
- Creación de la Clave de Seguridad: Se crea una clave de seguridad (securityKey) utilizando la clave secreta que se encuentra parametrizada en base de datos. Esta clave se utiliza para firmar el token y verificar su autenticidad posteriormente.

- Creación de Credenciales de Firma: Se crean las credenciales de firma (signingCredentials) utilizando la clave de seguridad y el algoritmo de firma HmacSha256. Estas credenciales se utilizan al generar la firma del token.

- Creación de ClaimsIdentity: Se crea un ClaimsIdentity que representa la identidad del usuario asociado al token. En este caso, se incluye un claim que contiene el nombre de usuario. **Nota:** *Un claim es una afirmación sobre el usuario que se agrega al token.*

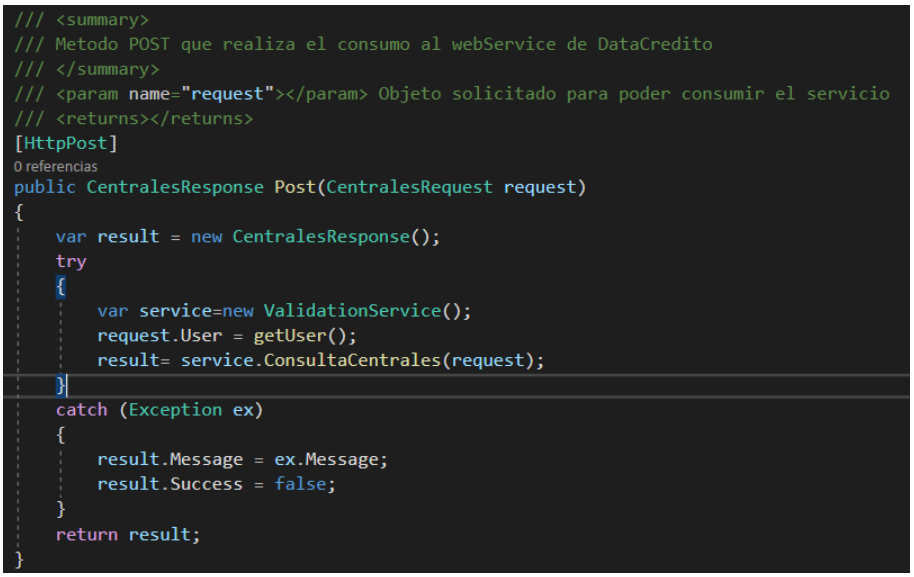
Estas líneas de código son esenciales para la creación segura de un JWT, ya que establecen los parámetros clave, la clave de seguridad para la firma, las credenciales de firma y la identidad del usuario asociada al token. Esta funcionalidad es crucial para garantizar la autenticidad y la integridad del JWT generado.

Si el JWT token fue creado correctamente este se envía al usuario como respuesta para el posterior uso de este. La API fue desarrollada para que todas las solicitudes que tuvieran la ruta */centrales* se deba recibir el token de autorización en la solicitud.



```
namespace Datacredito.Integrator.Api.Controllers
{
    [Authorize]
    [Route("centrales")]
    0 referencias
    public class CentralesController ...
}
```

De esta manera cada vez que se recibe una solicitud, GET o POST, este pasara por el filtro de Authorize que será el encargado de hacer ingeniería inversa y validar que el JWT recibido este creado por la misma aplicación y el tiempo de expiración no allá excedido su límite.



```
/// <summary>
/// Metodo POST que realiza el consumo al webService de DataCredito
/// </summary>
/// <param name="request"></param> Objeto solicitado para poder consumir el servicio
/// <returns></returns>
[HttpPost]
0 referencias
public CentralesResponse Post(CentralesRequest request)
{
    var result = new CentralesResponse();
    try
    {
        var service=new ValidationService();
        request.User = getUser();
        result= service.ConsultaCentrales(request);
    }
    catch (Exception ex)
    {
        result.Message = ex.Message;
        result.Success = false;
    }
    return result;
}
```

El método POST de la aplicación es la encargada de consumir el servicio web de Datacrédito, en la clase de *service.ConsultaCentrales s*erá la encargada de configurar la conexión como lo es el tipo de protocolo en que se comunicara, los certificados y claves para poder realizar el consumo y demás validaciones que requiere para comunicarse correctamente al servicio ofrecido por Datacrédito. Así mismo, la clase directamente hará la solicitud para obtener el historial crediticio correspondiente al dato recibido en el request, leerá y procesará la información obtenida y antes de enviarla como respuesta a la solicitud la guardará en la tabla llamada [*tbl_consulta_centrales]* con la respuesta encriptada.

Script para ver las consultas guardadas hechas por el API.

```
SELECT TOP (5) [id_consultacentrales]
      ,[fk_id_app]
      ,[tipo_identificacion]
      ,[numero_identificacion]
      ,[apellido_digitado]
      ,[fecha_consulta]
```

```
      ,[parametros_consulta]
      ,[respuesta]
      ,[codigo_respuesta]
      ,[consulta_exitosa]
      ,[id_origen]
  FROM [C10_Datacredito].[dbo].[tbl_consulta_centrales] order by id_consultacentrales desc
```

	id_consultacentrales	fk_id_app	tipo_identificacion	numero_identificacion	apellido_digitado	fecha_consulta	parametros_consulta	respuesta	codigo_respuesta	consulta_exitosa
1	629819	1	1	32184786	Guisao	2023-11-20 20:59:48.047	{"solicitud":{"clave":"89UPQ","ide...	kDWkwuLrgJHIfi/fx4shx+WChJwGrAgUVrUTP...	13	1
2	629818	1	1	1020840701	Gomez	2023-11-20 20:45:48.250	{"solicitud":{"clave":"89UPQ","ide...	kDWkwuLrgJHIfi/fx4shx+WChJwGrAgUVrUTP...	13	1
3	629817	1	1	40088154	Ramirez	2023-11-20 20:41:49.887	{"solicitud":{"clave":"89UPQ","ide...	kDWkwuLrgJHIfi/fx4shx+WChJwGrAgUVrUTP...	13	1
4	629816	1	1	80069005	Pineda	2023-11-20 20:39:39.470	{"solicitud":{"clave":"89UPQ","ide...	kDWkwuLrgJHIfi/fx4shx+WChJwGrAgUVrUTP...	13	1
5	629815	1	1	1098807739	Muñoz	2023-11-20 20:34:01.333	{"solicitud":{"clave":"89UPQ","ide...	kDWkwuLrgJHIfi/fx4shx+WChJwGrAgUVrUTP...	13	1

Al saber que la respuesta obtenida es información financiera y tener datos sensibles de los clientes de Credi10, se decidió cifrar la respuesta mediante la encriptación AES 256 que es la encargada de proteger la información utilizando una clave de 256 bits para cifrar y descifrar la información.

```
/// <summary>
/// Metodo para obtener las diferentes consultas realizadas a un numero de identificacion
/// </summary>
/// <param name="id"></param> Numero de identificacion del cliente
/// <returns></returns>
[HttpGet]
[Route("centrales/{id}")]
0 referencias
public List<ConsultaResponse> Get(string id)
{
    var bo=new BusinessLog();
    return bo.List(id);
}
```

El método GET de la aplicación recibe en su solicitud un id que será igual a el número de identificación de la persona a la que quiere consultar, este método devolverá todas las solicitudes realizadas y guardadas en la base de datos de la persona con información del consumo y el historial crediticio que tuvo en cada consulta.

1.3.5 Lista de Programas, Arhivos, Paquetes, Elementos BD

[C10_Datacredito].[tbl_apps]

Tabla dentro del esquema C10_Datacredito, que permite almacenar los usuarios que se autorizarán para consumir los métodos del API.

[C10_Datacredito].[tbl_consulta_centrales]

Tabla dentro del esquema C10_Datacredito, el cual permite almacenar todas las solicitudes realizadas a la API y la respuesta que obtuvo cada solicitud realizada.

[C10_Datacredito].[tbl_parametros]

Tabla dentro del esquema C10_Datacredito, el cual permite almacenar y obtener en la aplicación los diferentes parámetros que permiten el correcto consumo al servicio web Historia de crédito.

[C10_Datacredito].[tbl_codigos_consulta]

Tabla dentro del esquema C10_Datacredito, el cual permite almacenar y validar los tipos de respuesta que se pueden obtener desde el servicio web Historia de crédito. Esto para una mayor comprensión con la respuesta obtenida.

1.3.6 Lista de Scripts y la descripción de lo que contiene cada uno

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Originacion_API_Datacredito_Creacion_Tablas.sql

Script 2. Inserción de datos en la tabla tbl_codigos_consulta Contiene la inserción de datos de los códigos de consulta que corresponde a los códigos y la descripción que se puede obtener en el consumo al servicio web de historia de crédito. NOTA: Al pasar el tiempo pueden irse creando nuevos códigos de respuesta

Originacion_API_Datacredito_Insertar_Datos_Iniciales.sql

1.4. Observaciones y/o Comentarios

Para hacer pruebas de conexión y validar el consumo correcto al producto ofrecido por data crédito se debe consumir desde SOAP UI, para poder hacerlo se deben seguir primero unos pasos en la aplicación para configurar la conexión, usuarios y llaves y así poder consumir el servicio WEB.

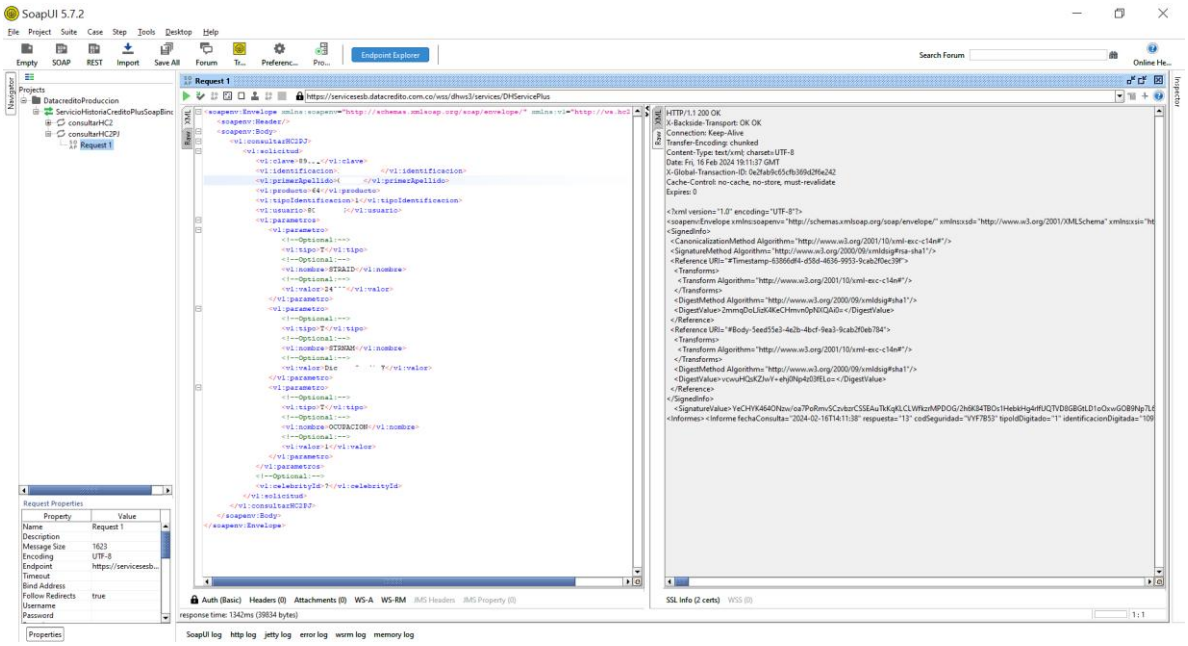
En él Manual Configuracion SOAP consumo historial crediticio se encontrará el paso a paso necesario para poder configurar el consumo.

En este punto se necesita tener una llave .p12 y la contraseña para realizar la configuración, como nombres y claves de usuario. Esta información puede actualizarse cada año y sus valores podrán ser encontrados en los parámetros de la base de datos.

Nota: La llave debe estar guardada en el equipo donde se está realizando la prueba. Para obtenerla consulta con el equipo informático de credi10.

Para realizar la prueba se podrá exportar desde soap el siguiente proyecto ProyectoSOAP después de finalizar la configuración inicial de SOAP UI.

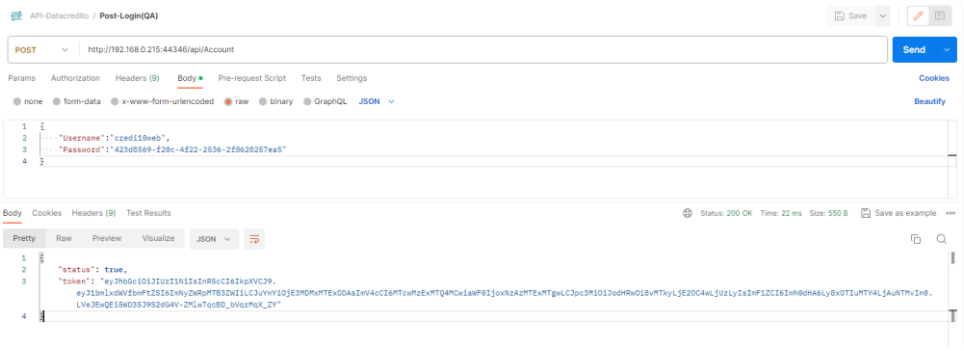
- **Consumo SOAP UI**


SoapUI 5.7.2
File Project Suite Case Step Tools Desktop Help
Empty SOAP REST Import Save All Forum Endpoint Explorer
Search Forum
Projects
DatacreditoProduccion
ServicioHistoriaCreditoPlusSoapBinc
consultarHC2
consultarHC2PJ
Request 1
Request Properties
Property Value
Name Request 1
Description
Message Size 1623
Encoding UTF-8
Endpoint https://servicesesb...
Timeout
Bind Address
Follow Redirects true
Username
Password
Properties
Request 1
https://servicesesb.datacredito.com.co/wss/dhws3/services/DHServicePlus
Auth (Basic) Headers (0) Attachments (0) WS-A WS-RM JMS Headers JMS Property (0)
response time: 1342ms (39834 bytes)
SSL Info (2 certs) WSS (0)
SoapUI log http log jetty log error log wsrm log memory log

2. CASOS DE PRUEBAS UNITARIAS

A continuación, se demuestra las pruebas funcionales hechas desde POSTMAN.

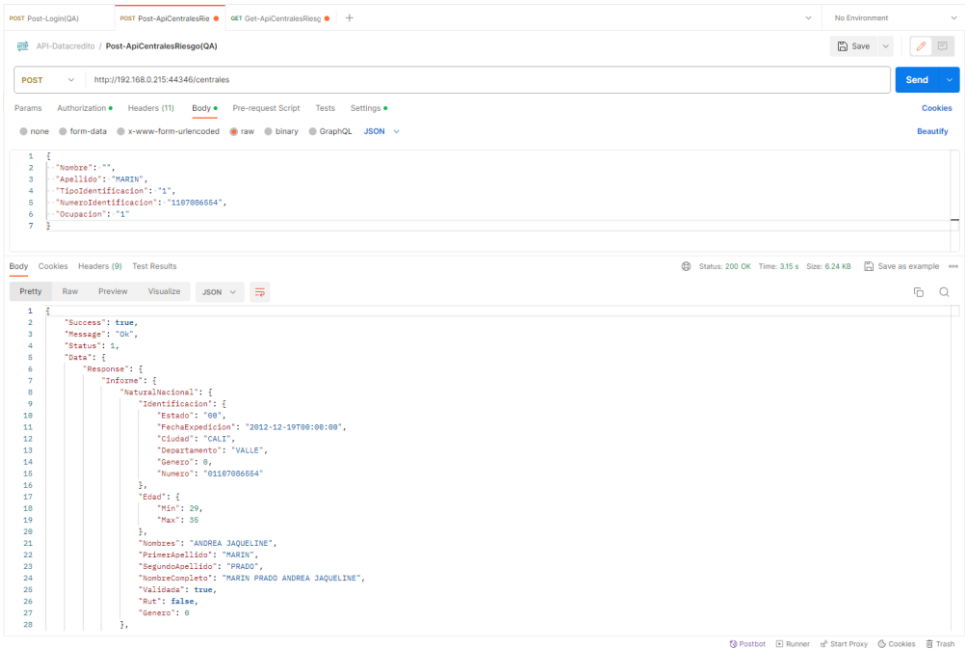
1. *Primero se debe realizar el consumo para autenticar el usuario con el API. La solicitud debe ser hecha mediante POST, y en el cuerpo de la solicitud se debe enviar un username y password que serán dados a los usuarios (diferentes aplicaciones) para autorizar su consumo.*



El curl de la solicitud es el siguiente:

```
curl --location 'http://192.168.0.xxx:xxxxx/api/Account' \
--header 'Content-Type: application/json' \
--data '{
    "Username":"xxxxxxxxx",
    "Password":"xxxxxxxxx"
}'
```

2. *Si en la primera solicitud nos da un status 200, nos devolverá un token que será el que debera utilizar para las siguientes solicitudes que se quieran realizar. En este caso se hara el consumo del POST que es el que consumira el servicio web de datacredito.*



En el cuerpo de la solicitud debe ir la información respectiva del cliente a quien se le quiere hacer el estudio financiero. En la respuesta se nos detallara toda la informacion del cliente y la decision con respecto a si se aprueba el cupo o no.

El curl de la solicitud es el siguiente:

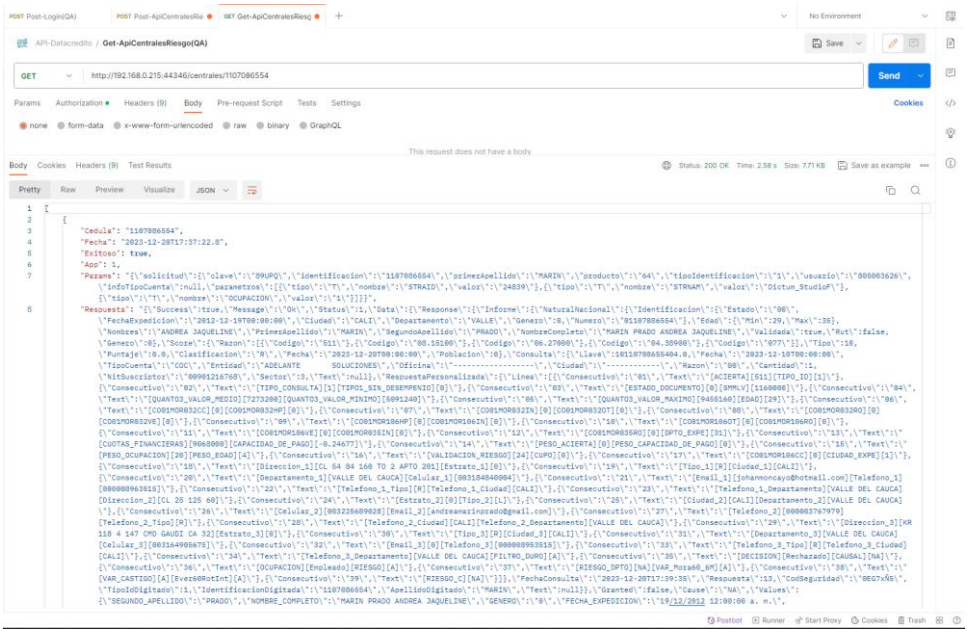
```
curl --location 'http://192.168.0.xxx:xxxx/centrales' \
--header 'app-user: xxxx' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImNyZWRpMTB3ZWIiLCJuYmYiOjE3MDMxMTE4MjYsImV4cCI6MTcwMzExMjEyNiwiaWF0IjoxNzAzMTExODI2LCJpc3MiOiJodHRwOi8vMTkyLjE2OC4wLjUzLyIsImF1ZCI6Imh0dHA6Ly8xOTIuMTY4LjAuNTMvIn0.tyXhSaPG0bvs_5CoI2wSbGOKh9sx5XQYEe8jah5c0LU' \
--data '{
  "Nombre": "",
  "Apellido": "MARIN",
  "TipoIdentificacion": "1",
  "NumeroIdentificacion": "1107086554",
  "Ocupacion": "1"
}'
```

Interpretación respuesta

A continuación, se describen los campos o columnas presentadas en el json de respuesta

- Success: Boleano que identifica si el proceso de consulta finalizo correctamente.
- Message: Devuelve mensaje informativo de la respuesta.
- Status: Entero que identifica si el consumo del API fue exitoso o no
- Data: Objeto con información del estudio crediticio de una cedula, el cual contiene información confidencial del cliente, el tipo de respuesta que se obtuvo del web service, la decisión de aprobado o rechazado, la causal de la decisión y el valor en caso de que se le apruebe un cupo.

3. La otra solicitud disponible es un GET el cual consulta todas las solicitudes hechas mediante el API a una cedula determinada.



El curl de la solicitud es la siguiente:

curl --location 'http://192.168.0.xxx:xxxxx/centrales/1107086554' \
--header 'app-user: credi10web' \
--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImNyZWRpMTB3ZWIiLCJuYmYiOjE3MDMxMTE4MjYsImV4cCI6MTcwMzExMjEyNiwiaWF0IjoxNzAzMTExODI2LCJpc3MiOiJodHRwOi8vMTkyLjE2OC4wLjUzLyIsImF1ZCI6Imh0dHA6Ly8xOTIuMTY4LjAuNTMvIn0.tyXhSaPG0bvs_5CoI2wSbGOKh9sx5XQYEe8jah5c0LU'

ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Autenticacion al API	Devolucion de un JWT	SI

2	Estudio Crediticio de una cedula	Devolucion estudio y decision	SI
3	Consulta de solicitudes a una cedula	Devolucion lista de solicitudes realizadas	SI

Recurso para realizar pruebas por Postman

- Originacion_API_Datacredito.postman_collection